CSE/ISyE 6740
Computational Data Analysis

Recurrent Neural Networks

11/4/2026

Kai Wang, Assistant Professor in Computational Science and Engineering
kwang692@gatech.edu

Georgia Tech

Outline

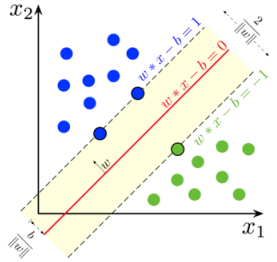
- **Advanced topics**
  - Recurrent neural network (RNN)
    - Sequence data
    - RNN
    - LSTM
    - Other RNN architectures

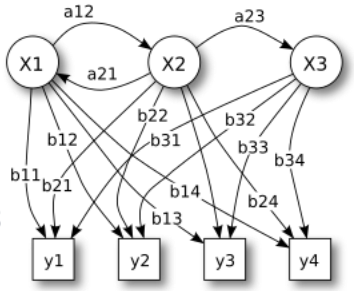
Georgia Tech

History of Deep Learning Models

Before 2012

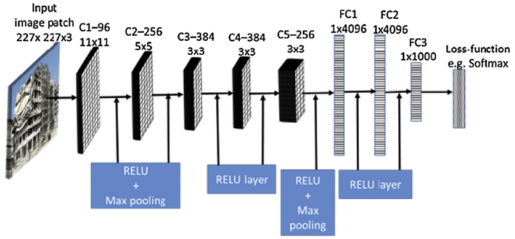
SVM / HMM /





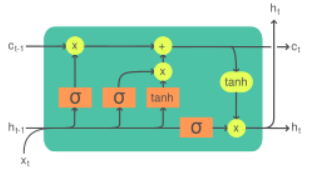
2012 ~ 2015

First deep CNN model

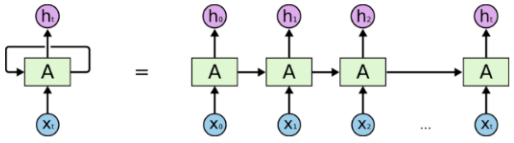


???? - 2017

LSTM / RNN



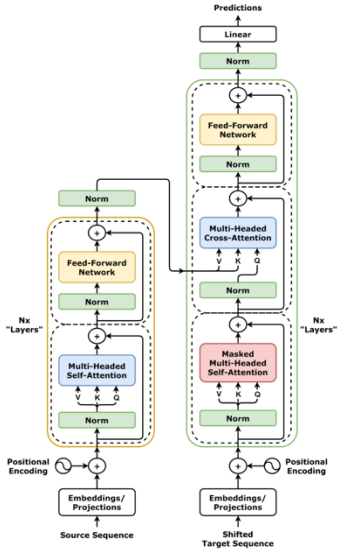




An unrolled recurrent neural network.

2017 - now

Transformer



???

Georgia Tech

Sequence Data

Georgia Tech

Sequence Data

- Each non-sequence data is composed of
  - Feature $x_t \in \mathbb{R}^d$
  - Label $y_t \in \mathbb{R}$



- What if the data is intrinsically a sequence?
  - Language / words
  - Stock market prices within a time period
  - Music / song





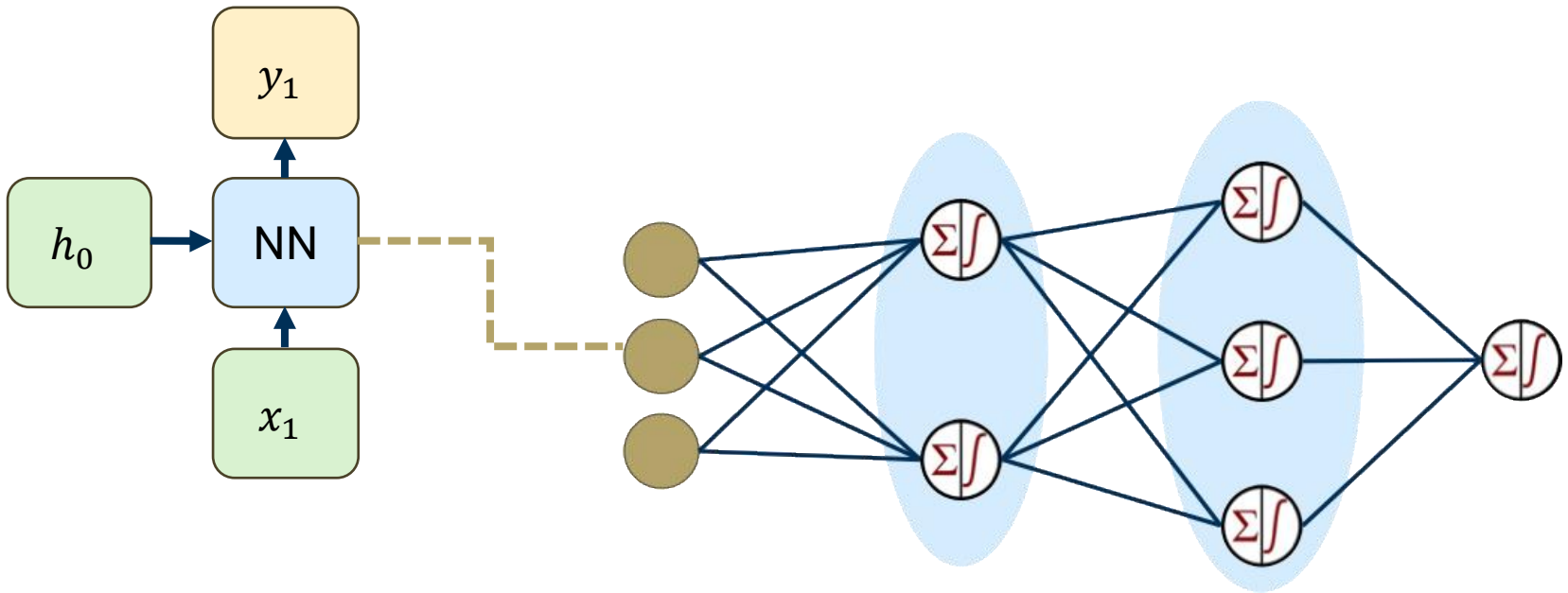
- Each sequence data is composed of
  - Feature $x_t \in \mathbb{R}^d$ for $t = 1,2,\dots,T$
  - Label $y_t \in \mathbb{R}$ for $t = 1,2,\dots,T$

Georgia Tech

Recurrent Neural Network

- Recurrent neural network (RNN) is a neural network architecture designed to handle sequential data

Outputs $y_t \in \mathbb{R}$



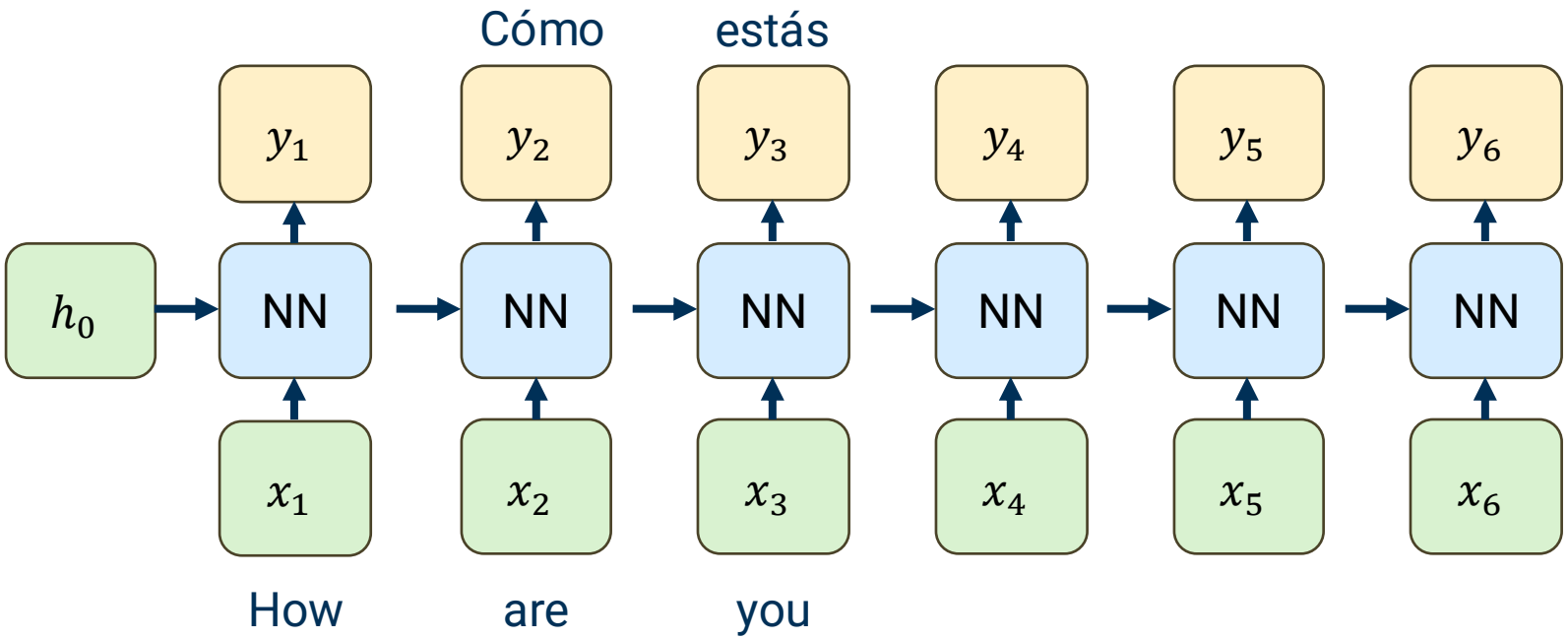
Inputs $x_t \in \mathbb{R}^d$

Georgia Tech

Recurrent Neural Network

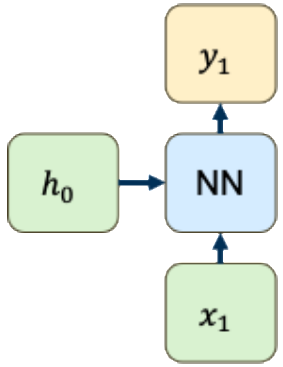
- Recurrent neural network (RNN) is a neural network architecture designed to handle sequential data

Outputs y_t: machine translation, next token prediction, sentiment, etc.

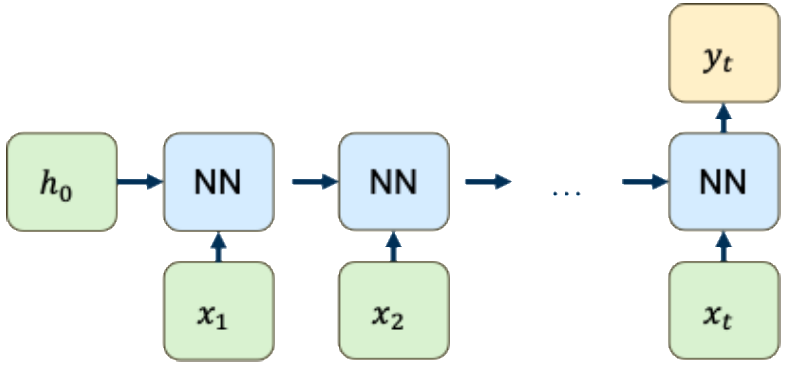


Inputs $x_t \in \mathbb{R}^d$: token embedding

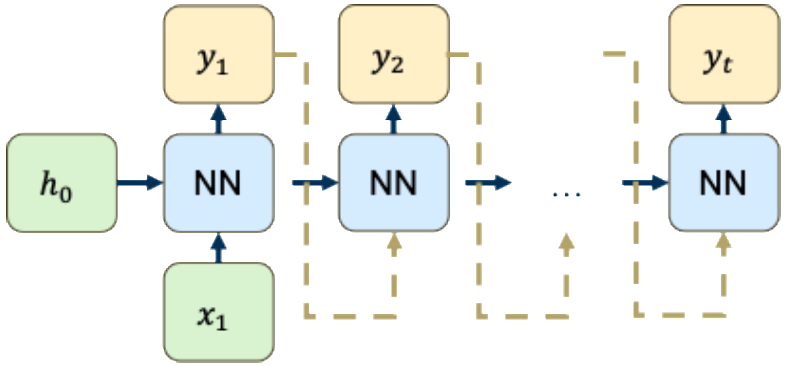
RNN for Different Sequence Data



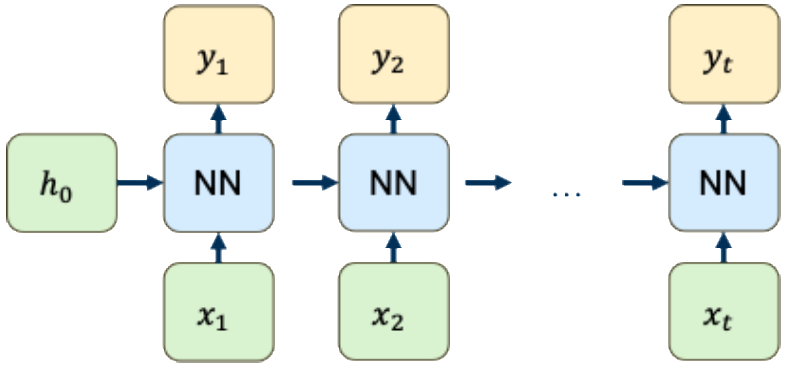
One-to-one: traditional neural networks



Many-to-one: sentiment analysis



One-to-many: autoregressive models (LLMs)



Many-to-many: machine translation

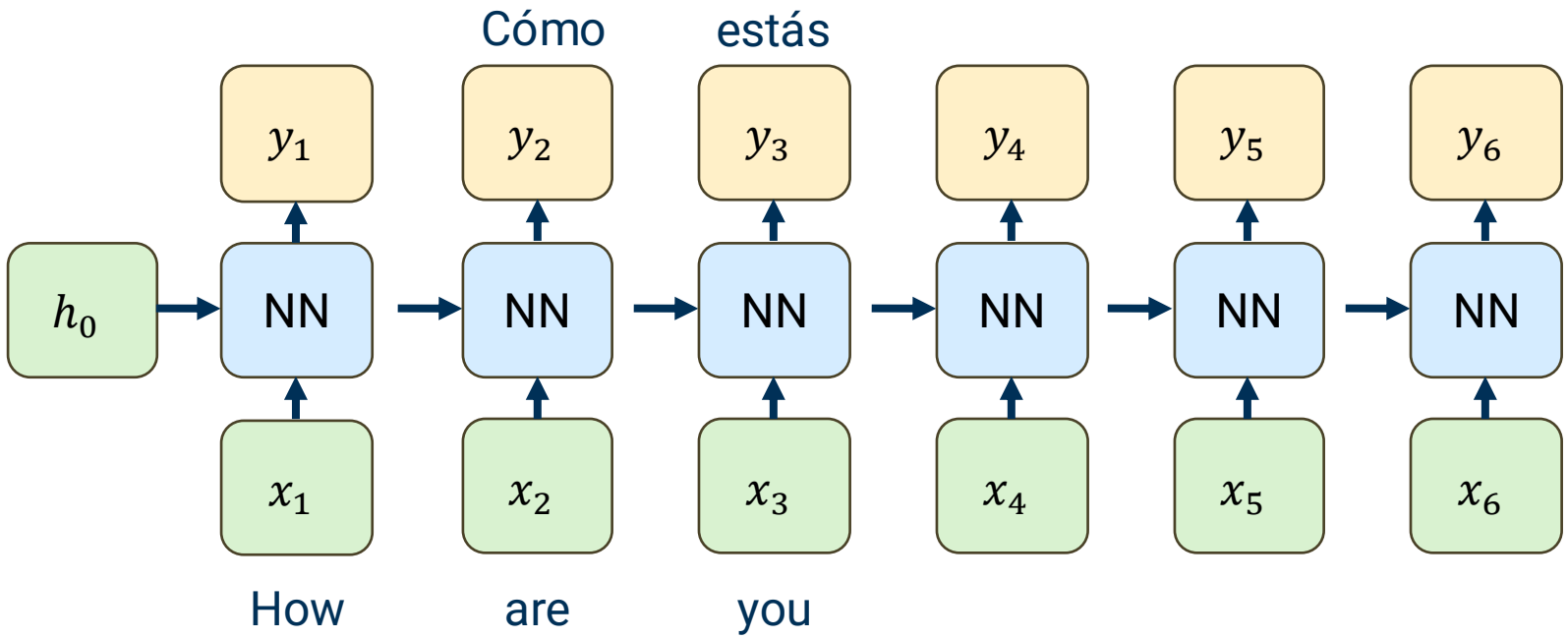
Georgia Tech

Recurrent Neural Network

Georgia Tech

Recurrent Neural Network

- How does this work specifically?



Georgia Tech

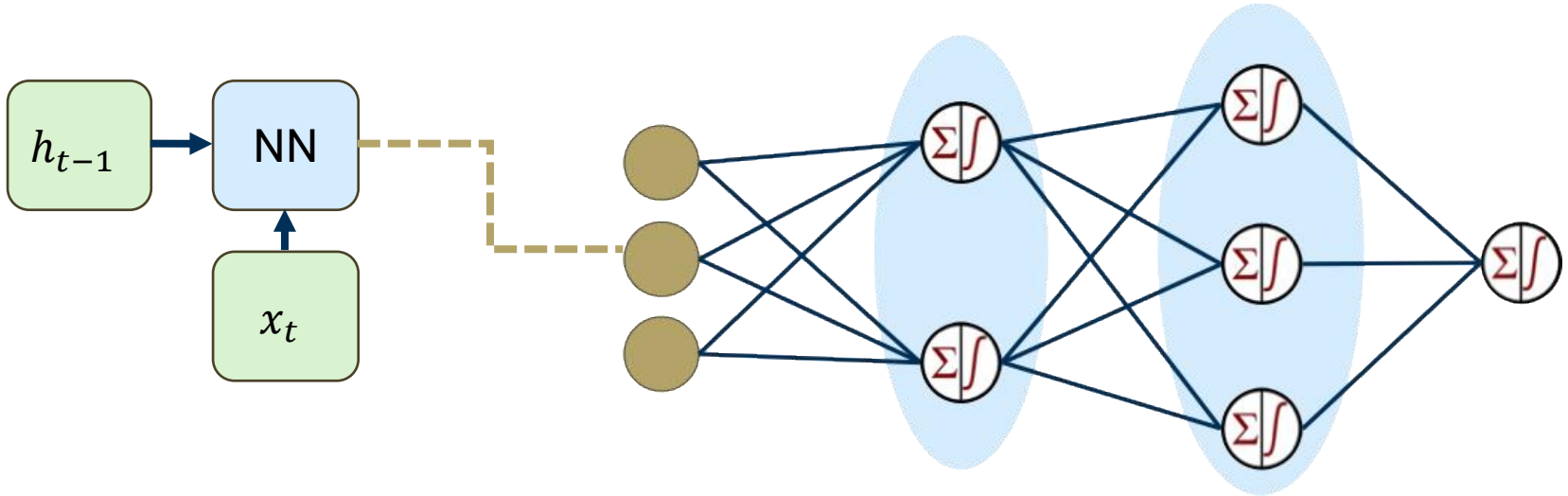
Recurrent Neural Network

- RNN maintains a recurrent MLP
- Each recurrence processes the hidden h_{t-1} and the new feature x_t to generate h_t

Outputs $y_t \in \mathbb{R}$

```python
# Efficient implementation equivalent to the following with bidirectional=False
rnn = nn.RNN(input_size, hidden_size, num_layers)
params = dict(rnn.named_parameters())
def forward(x, hx=None, batch_first=False):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(rnn.num_layers, batch_size, rnn.hidden_size)
    h_t_minus_1 = hx.clone()
    h_t = hx.clone()
    output = []
    for t in range(seq_len):
        for layer in range(rnn.num_layers):
            input_t = x[t] if layer == 0 else h_t[layer - 1]
            h_t[layer] = torch.tanh(
                input_t @ params[f"weight_ih_l{layer}"].T
                + h_t_minus_1[layer] @ params[f"weight_hh_l{layer}"].T
                + params[f"bias_hh_l{layer}"]
                + params[f"bias_ih_l{layer}"]
            )
        output.append(h_t[-1].clone())
        h_t_minus_1 = h_t.clone()
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

PyTorch RNN implementation



Vanilla version

$$= f_W(h_{t-1}, x_t)$$
$$= tanh(W_x x_t + W_h h_{t-1} + b_h)$$

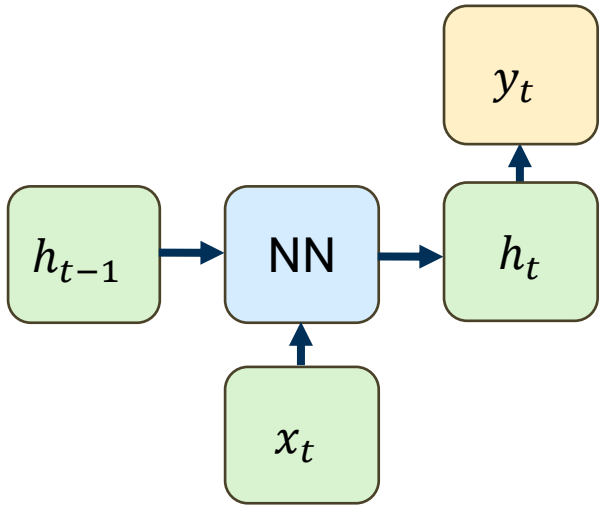
Inputs $x_t \in \mathbb{R}^d$

Georgia Tech

Recurrent Neural Network

- After the new hidden state h_t is generated, we use h_t to generate y_t

Outputs $y_t \in \mathbb{R}$



$= f_{W_y}(h_t) = W_y h_t + b_y$

Inputs $x_t \in \mathbb{R}^d$

```python
# Efficient implementation equivalent to the following with bidirectional=False
rnn = nn.RNN(input_size, hidden_size, num_layers)
params = dict(rnn.named_parameters())
def forward(x, hx=None, batch_first=False):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(rnn.num_layers, batch_size, rnn.hidden_size)
    h_t_minus_1 = hx.clone()
    h_t = hx.clone()
    output = []
    for t in range(seq_len):
        for layer in range(rnn.num_layers):
            input_t = x[t] if layer == 0 else h_t[layer - 1]
            h_t[layer] = torch.tanh(
                input_t @ params[f"weight_ih_l{layer}"].T
                + h_t_minus_1[layer] @ params[f"weight_hh_l{layer}"].T
                + params[f"bias_hh_l{layer}"]
                + params[f"bias_ih_l{layer}"]
            )
        output.append(h_t[-1].clone())
        h_t_minus_1 = h_t.clone()
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

PyTorch RNN implementation

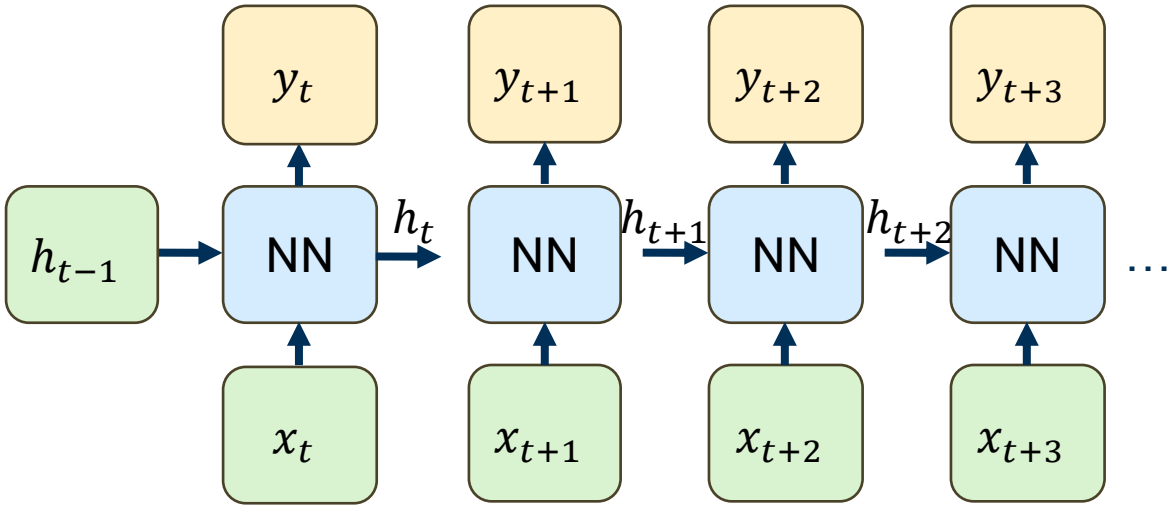
```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network

- Embedding h_t stores all the information up to t
- **Many-to-many:** repeat the same process for next inputs

Outputs $y_t \in \mathbb{R}$



Inputs $x_t \in \mathbb{R}^d$

```python
# Efficient implementation equivalent to the following with bidirectional=False
rnn = nn.RNN(input_size, hidden_size, num_layers)
params = dict(rnn.named_parameters())
def forward(x, hx=None, batch_first=False):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(rnn.num_layers, batch_size, rnn.hidden_size)
    h_t_minus_1 = hx.clone()
    h_t = hx.clone()
    output = []
    for t in range(seq_len):
        for layer in range(rnn.num_layers):
            input_t = x[t] if layer == 0 else h_t[layer - 1]
            h_t[layer] = torch.tanh(
                input_t @ params[f"weight_ih_l{layer}"].T
                + h_t_minus_1[layer] @ params[f"weight_hh_l{layer}"].T
                + params[f"bias_hh_l{layer}"]
                + params[f"bias_ih_l{layer}"]
            )
        output.append(h_t[-1].clone())
        h_t_minus_1 = h_t.clone()
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

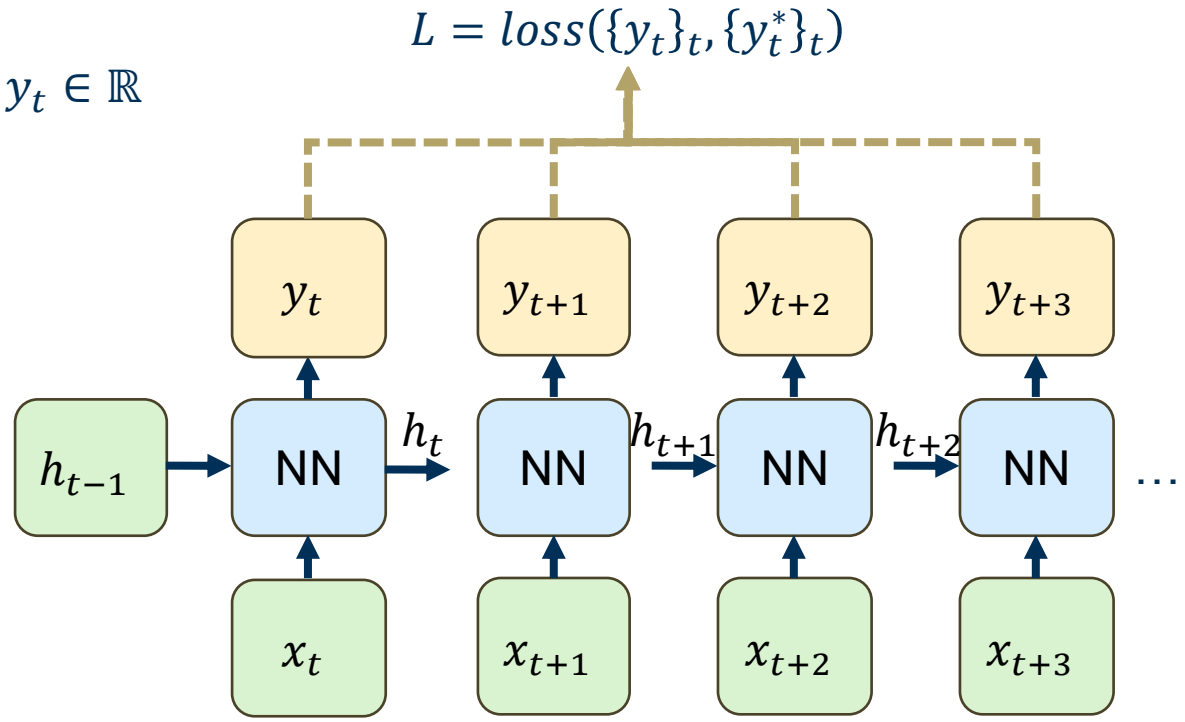
PyTorch RNN implementation

```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network

- Define the loss function to backprop and train the model




```python
# Efficient implementation equivalent to the following with bidirectional=False
rnn = nn.RNN(input_size, hidden_size, num_layers)
params = dict(rnn.named_parameters())
def forward(x, hx=None, batch_first=False):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(rnn.num_layers, batch_size, rnn.hidden_size)
    h_t_minus_1 = hx.clone()
    h_t = hx.clone()
    output = []
    for t in range(seq_len):
        for layer in range(rnn.num_layers):
            input_t = x[t] if layer == 0 else h_t[layer - 1]
            h_t[layer] = torch.tanh(
                input_t @ params[f"weight_ih_l{layer}"].T
                + h_t_minus_1[layer] @ params[f"weight_hh_l{layer}"].T
                + params[f"bias_hh_l{layer}"]
                + params[f"bias_ih_l{layer}"]
            )
        output.append(h_t[-1].clone())
        h_t_minus_1 = h_t.clone()
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

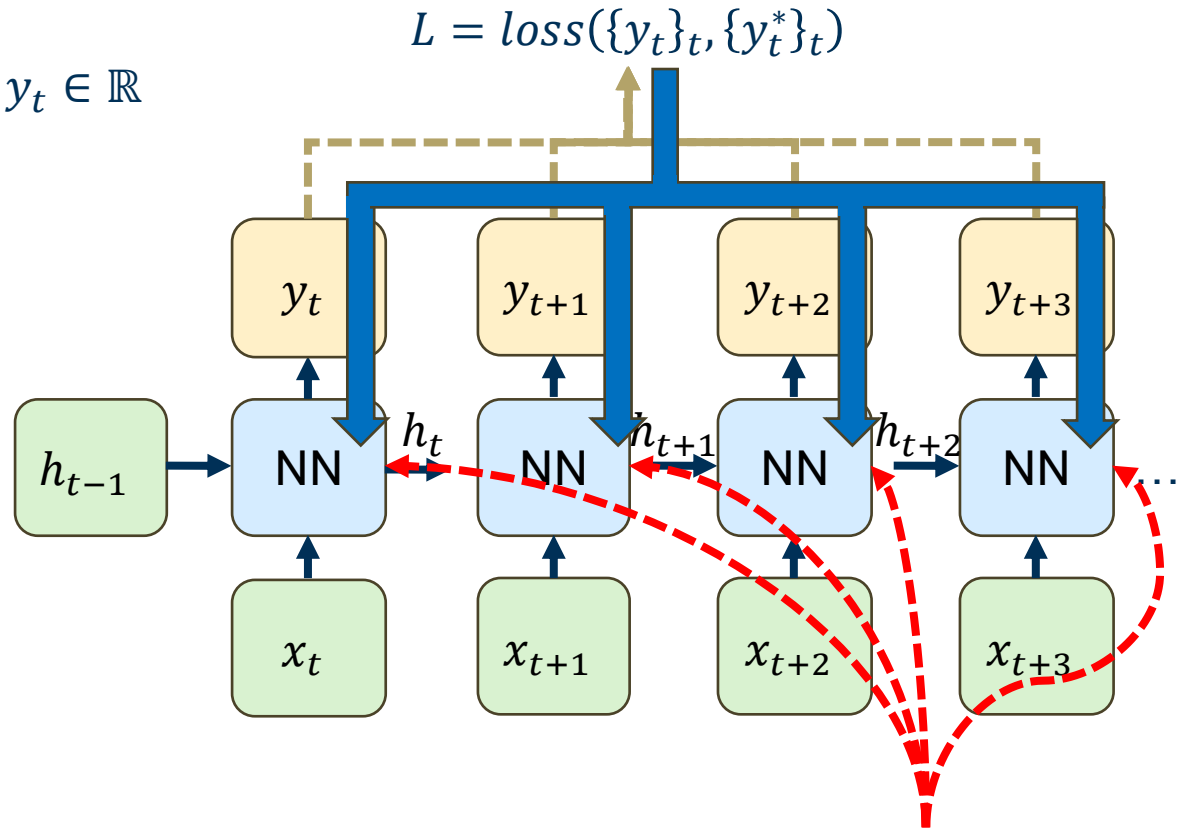
PyTorch RNN implementation

```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network Computation Graph

1. What are the trainable parameters?
2. How the backpropagation works?



$$\frac{dL}{dW} = \sum_{t=1}^{T} \frac{\partial L}{\partial y_t} \frac{dy_t}{dW}$$

$$= \sum_{t=1}^{T} \frac{\partial L}{\partial y_t} \left(\frac{\partial y_t}{\partial h_t} \frac{dh_t}{dW} + \frac{\partial y_t}{\partial W_y} \right)$$

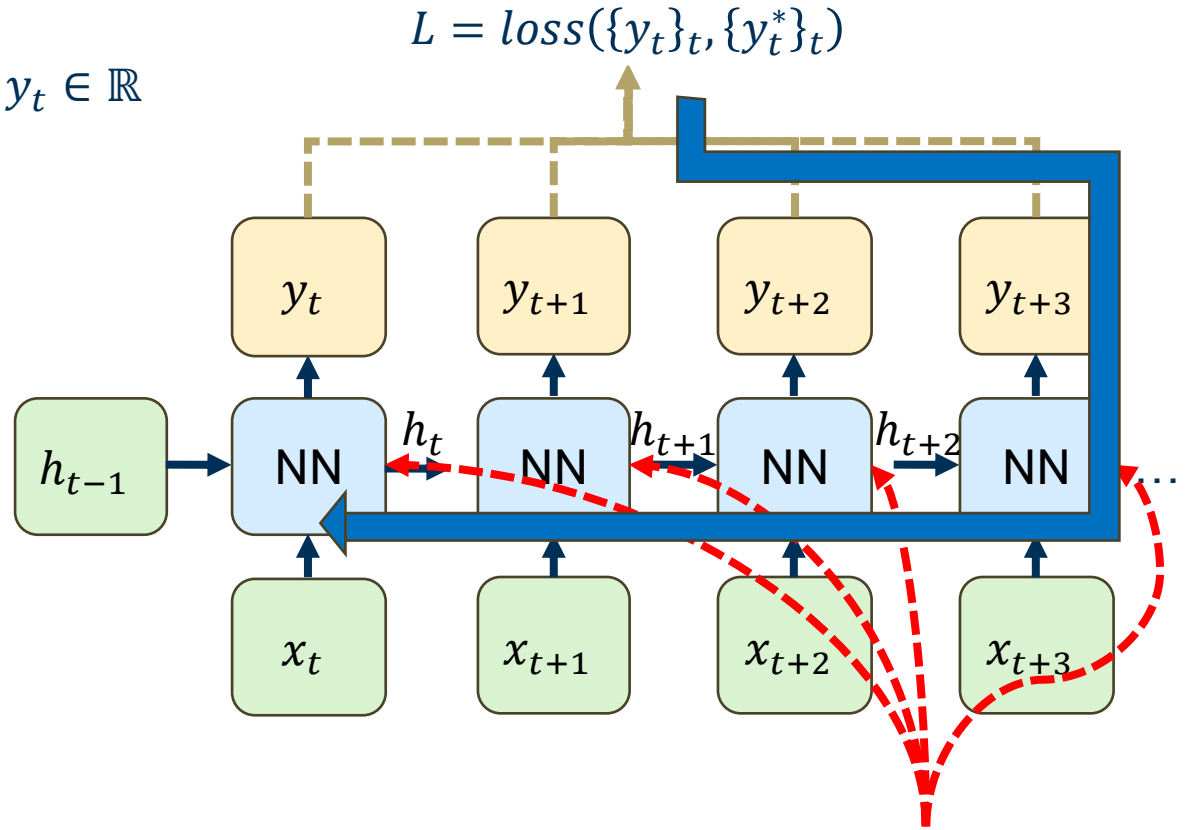
$$h_t = f_W(h_{t-1}, x_t)$$
$$= tanh(W_x x_t + W_h h_{t-1} + b_h)$$

$$\frac{dh_t}{dW} = \frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dW}$$
$$= \frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-1}}{dW}$$
$$= \cdots$$

Reuse the same weights $W = (W_x, W_h, W_y)$ at every time step

Recurrent Neural Network Computation Graph

1. What are the trainable parameters?
2. How the backpropagation works?



$$\frac{dh_t}{dW} = \frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}}\frac{dh_{t-1}}{dW}$$
$$= \frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}}\frac{dh_{t-1}}{dW}$$
$$= \cdots$$

We need to compute

Depends on eigenvalue

$$\frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}}\cdots\frac{\partial h_1}{\partial h_0} = \left(\prod_{s=0}^{t-1} tanh'(h_s)\right)(W_h)^t$$

All almost always < 1

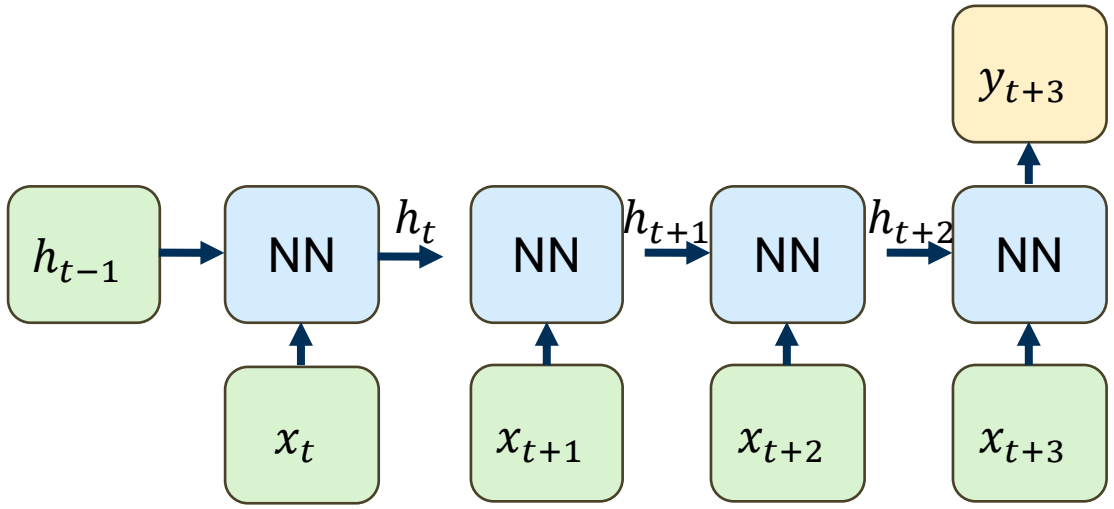
Gradient vanishing or exploding!!

Reuse the same weights $W = (W_x, W_h, W_y)$ at every time step

Recurrent Neural Network Computation Graph

- **Many-to-one:** only generate prediction for the last one

Outputs $y_t \in \mathbb{R}$



Inputs $x_t \in \mathbb{R}^d$

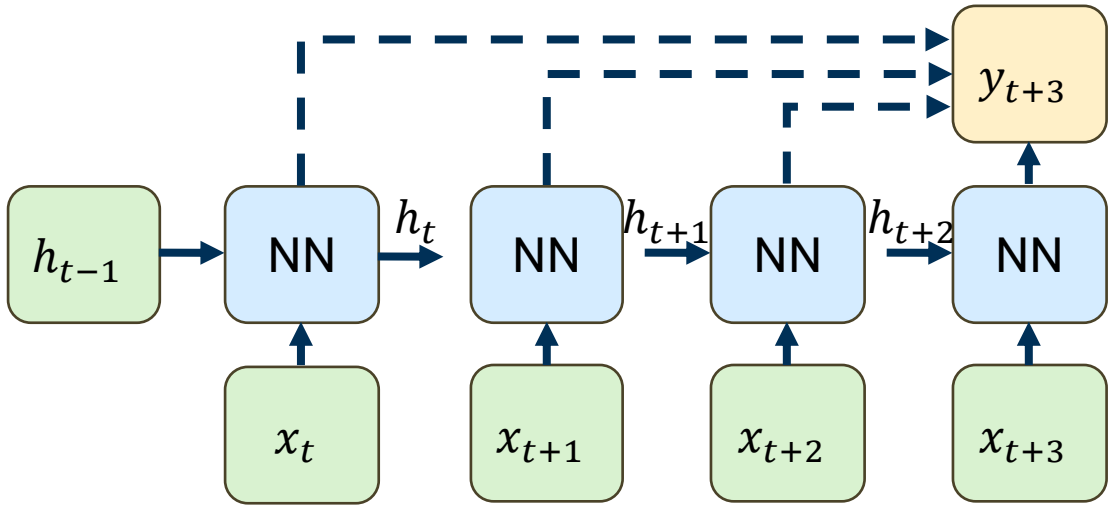
```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network Computation Graph

- **Many-to-one:** add dependency to previous hidden states

Outputs $y_t \in \mathbb{R}$



Inputs $x_t \in \mathbb{R}^d$

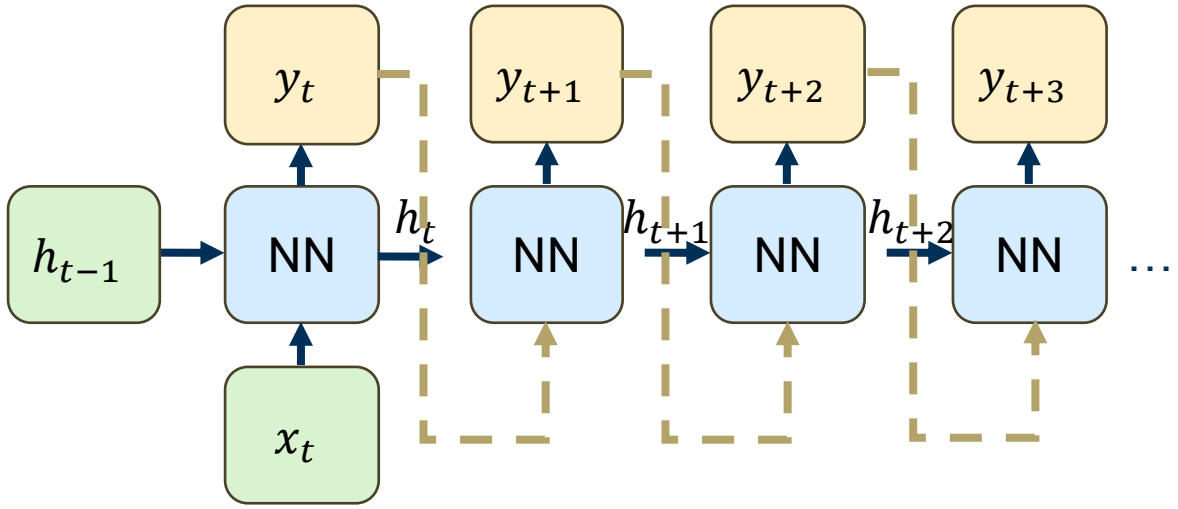
```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network Computation Graph

- **One-to-many:** autoregressive

Outputs $y_t \in \mathbb{R}$



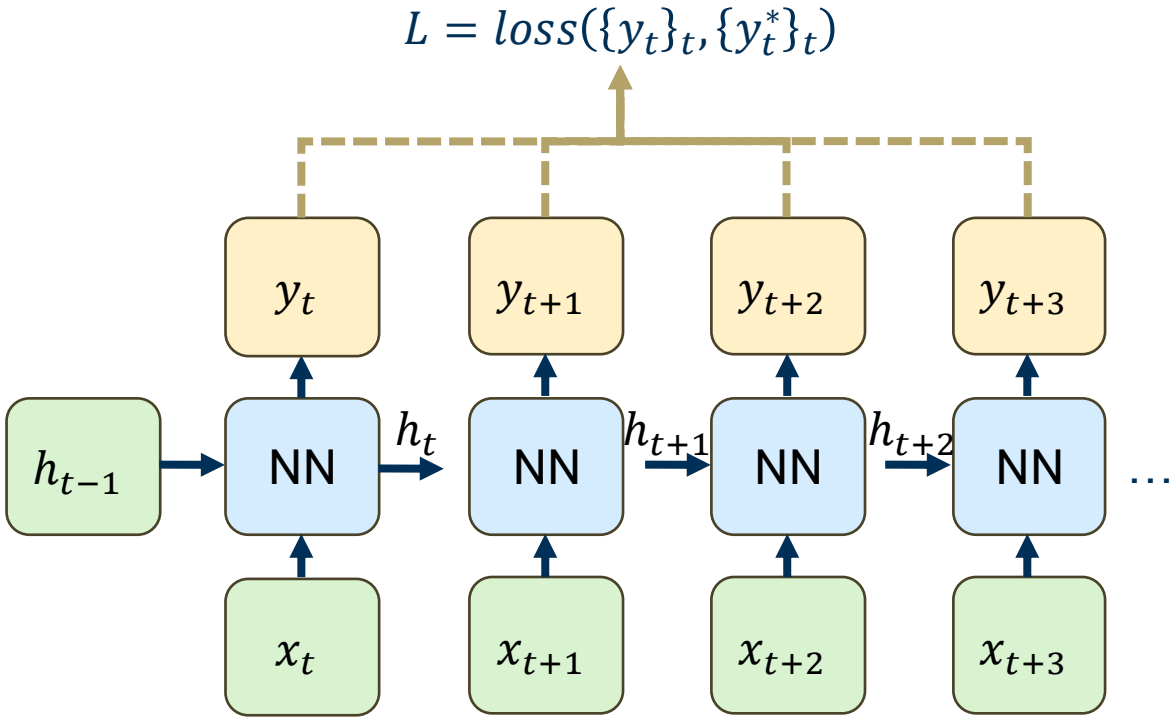
Inputs $x_t \in \mathbb{R}^d$

```python
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=8, hidden_dim=16, num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=False  # we'll keep (seq, batch, feat)
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (seq_len, batch)
        x = self.embed(x)  # (seq_len, batch, embed_dim)
        out, hidden = self.rnn(x, hidden)  # out: (seq_len, batch, hidden_dim)
        logits = self.fc(out)  # (seq_len, batch, vocab_size)
        return logits, hidden
```

Recurrent Neural Network Summary

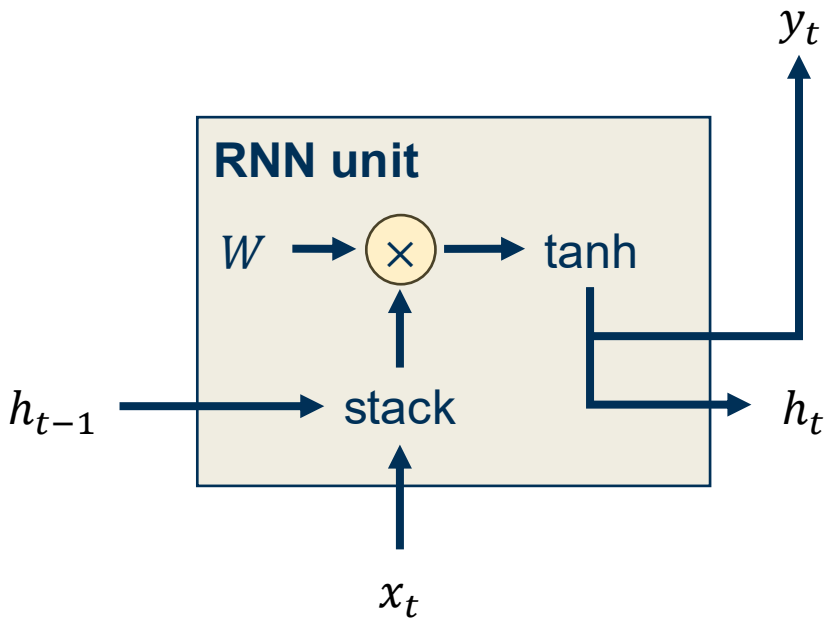
- Recurrently apply the same NN to new inputs x_t to generate hidden states h_t
- Use the hidden state(s) h_t to generate output(s)
- Unstable gradient in long sequences
  - Vanishing gradient if the eigenvalue < 1: $(W_h)^T \to 0$
  - Exploding gradient if the eigenvalue > 1: $(W_h)^T \to \infty$

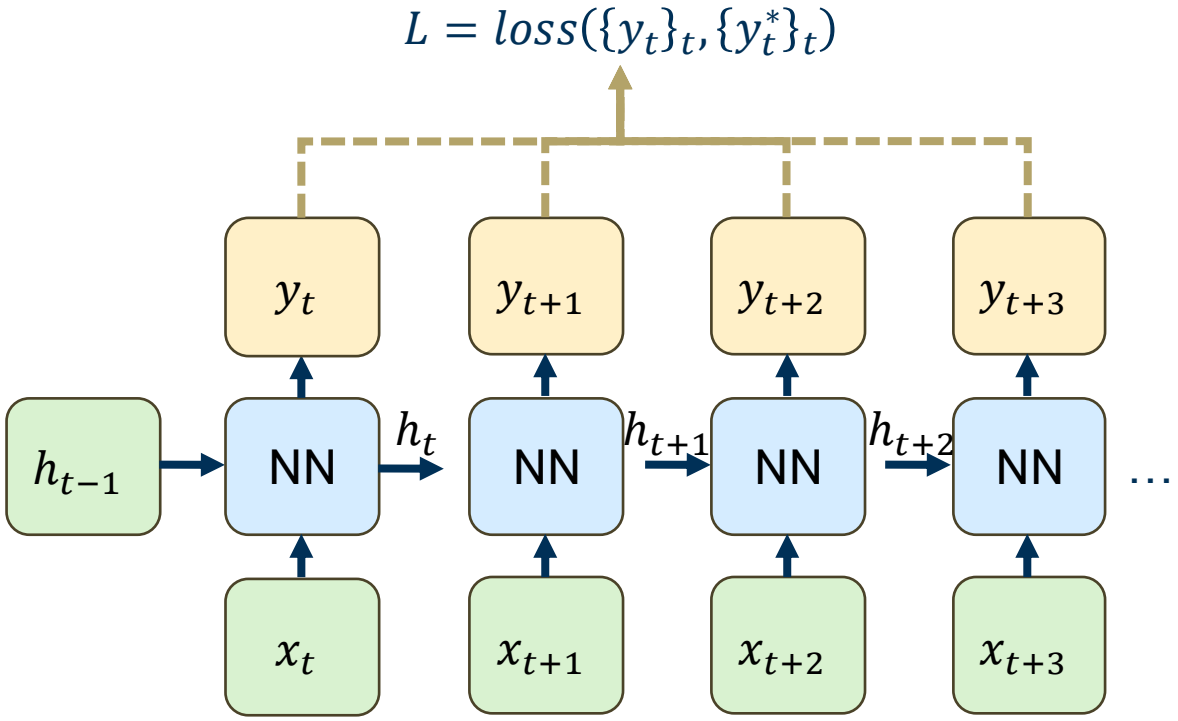


Georgia Tech

Recurrent Neural Network Summary

- Any solutions to handle long sequences?





Georgia Tech

Long Short-Term Memory

Georgia Tech

Hochreiter et al., 1997

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W\begin{pmatrix}h_{t-1}\\ x_t\end{pmatrix}\right)$$

LSTM

Four states →

$$\begin{pmatrix}i\\ f\\ o\\ g\end{pmatrix} = \begin{pmatrix}\sigma\\ \sigma\\ \sigma\\ tanh\end{pmatrix} W\begin{pmatrix}h_{t-1}\\ x_t\end{pmatrix}$$

Cell state →

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state →

$$h_t = o \odot \tanh(c_t)$$

Long short-term memory

S **Hochreiter**, J Schmidhuber - Neural computation, **1997** - ieeexplore.ieee.org

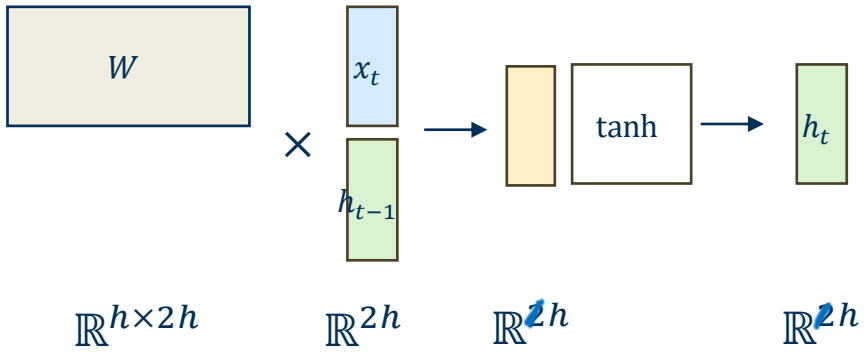
… (**LSTM**), a novel recurrent network architecture in conjunction with an appropriate gradient-based learning algorithm. **LSTM** is … We will see that **LSTM** uses multiplicative units (MUs) to …

☆ Save 99 Cite Cited by 136437 Related articles All 51 versions »

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$



LSTM

Four states →

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state →

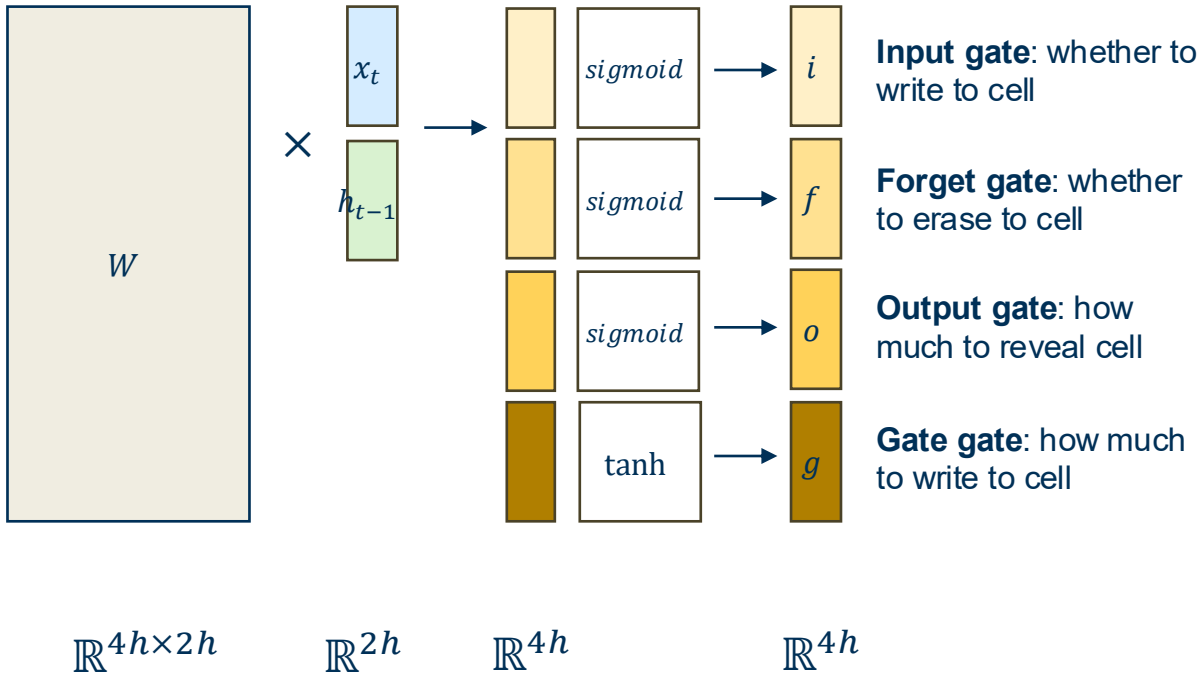
$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state →

$$h_t = o \odot \tanh(c_t)$$

Georgia Tech

Long Short Term Memory (LSTM)



Input gate: whether to write to cell

Forget gate: whether to erase to cell

Output gate: how much to reveal cell

Gate gate: how much to write to cell

LSTM

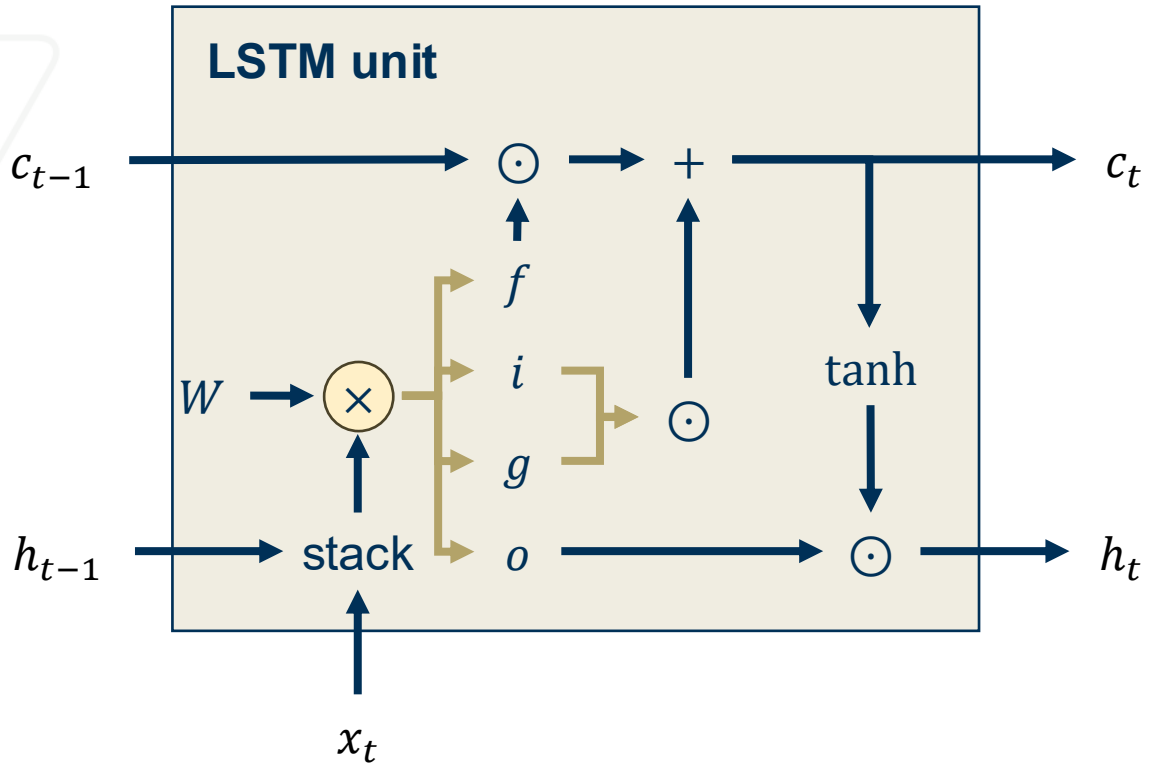
Four states → $\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$

Cell state → $c_t = f \odot c_{t-1} + i \odot g$

Hidden state → $h_t = o \odot \tanh(c_t)$

Georgia Tech

Long Short Term Memory (LSTM)



LSTM

Four states $\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$

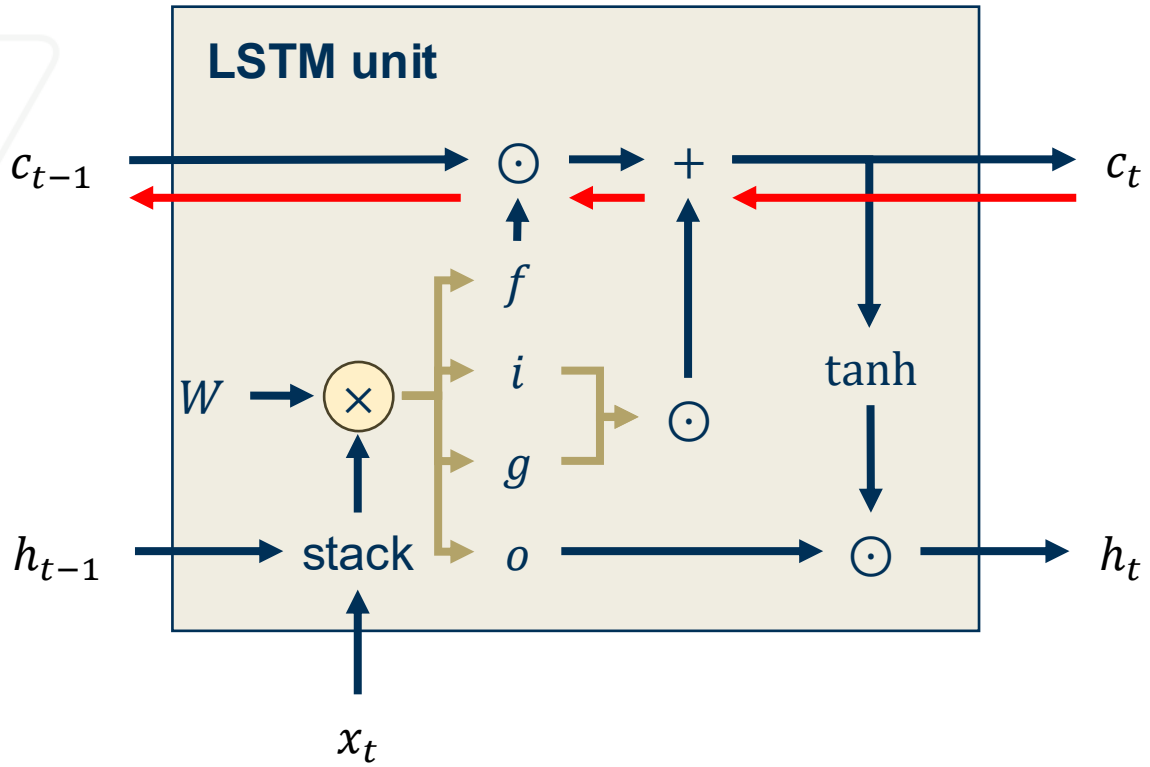
Cell state $c_t = f \odot c_{t-1} + i \odot g$

Hidden state $h_t = o \odot \tanh(c_t)$

Georgia Tech

LSTM Backpropagation

- How does the backpropagation work?
  - Backprop from c_t to c_{t-1} only involves elementwise multiplication $\odot$ and addition
  - No matrix multiplication



LSTM

Four states $\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$

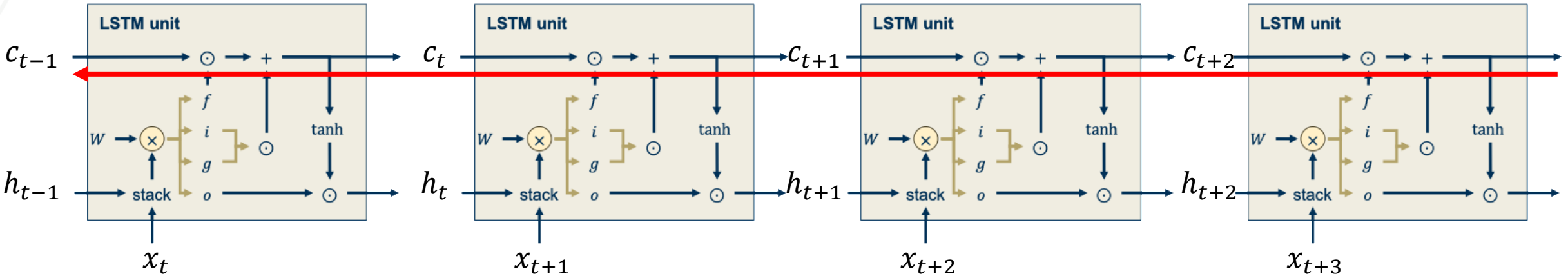
Cell state $c_t = f \odot c_{t-1} + i \odot g$

Hidden state $h_t = o \odot \tanh(c_t)$

Georgia Tech

LSTM Backpropagation

- When multiple LSTM units are connected, the gradient can flow through the cells uninterruptedly.
  - No matrix multiplication
  - No vanishing/exploding gradient issue in this route

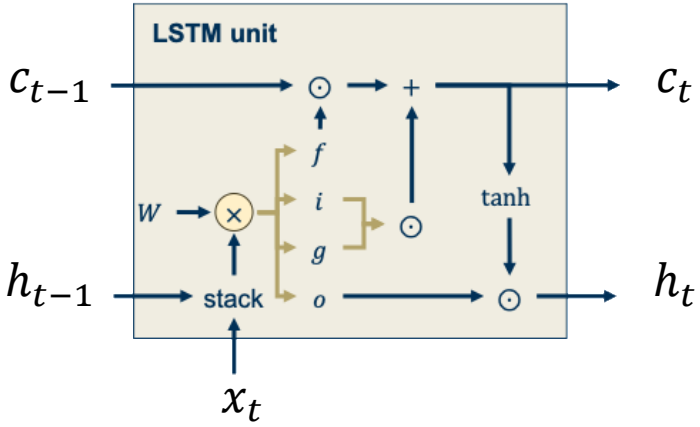


Georgia Tech

LSTM Discussion

- The LSTM architecture makes it easier for the RNN to preserve/erase information over multiple time steps
  - $f = 1 \Rightarrow$ cell information will be preserved
  - $f = 0 \Rightarrow$ cell information will be erased

- LSTM provides an easier route to backpropagate gradient
  - It doesn't guarantee other routes also have good gradient
  - Doesn't solve vanishing/exploding gradient issues
  - But at least there is one good route



Georgia Tech

Other RNN Architectures

Georgia Tech

Other RNN Architectures

Learning phrase representations using RNN encoder-decoder for **statistical machine translation**

K **Cho**, B Van Merriënboer, C Gulcehre… - arXiv preprint arXiv …, **2014** - arxiv.org

In this paper, we propose a novel neural network model called RNN Encoder-Decoder that consists of two recurrent neural networks (RNN). One RNN encodes a sequence of symbols into a fixed-length vector representation, and the other decodes the representation into another sequence of symbols. The encoder and decoder of the proposed model are jointly trained to maximize the conditional probability of a target sequence given a source sequence. The performance of a statistical machine translation system is empirically found …

☆ Save Cite Cited by 37276 Related articles All 24 versions »

GRU

class **torch.nn.GRU(*input_size*, *hidden_size*, *num_layers*=*1*, *bias*=*True*, *batch_first*=*False*, *dropout*=*0.0*, *bidirectional*=*False*, *device*=*None*, *dtype*=*None*)** [source]

Apply a multi-layer gated recurrent unit (GRU) RNN to an input sequence. For each element in the input sequence, each layer computes the following function:

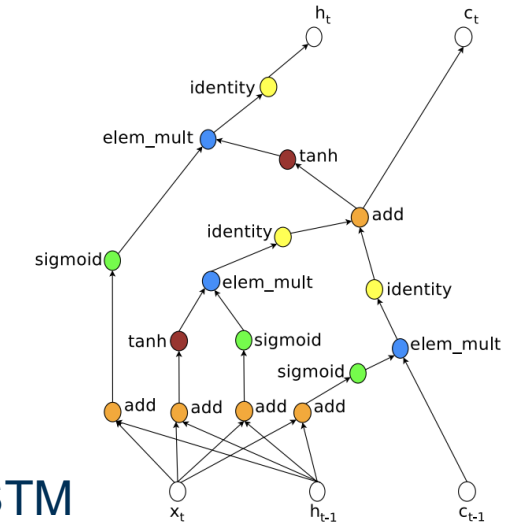
reset
update
new state

$$
\begin{aligned}
r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{(t-1)} + b_{hr}) \\
z_t &= \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{(t-1)} + b_{hz}) \\
n_t &= \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{(t-1)} + b_{hn})) \\
h_t &= (1 - z_t) \odot n_t + z_t \odot h_{(t-1)}
\end{aligned}
$$

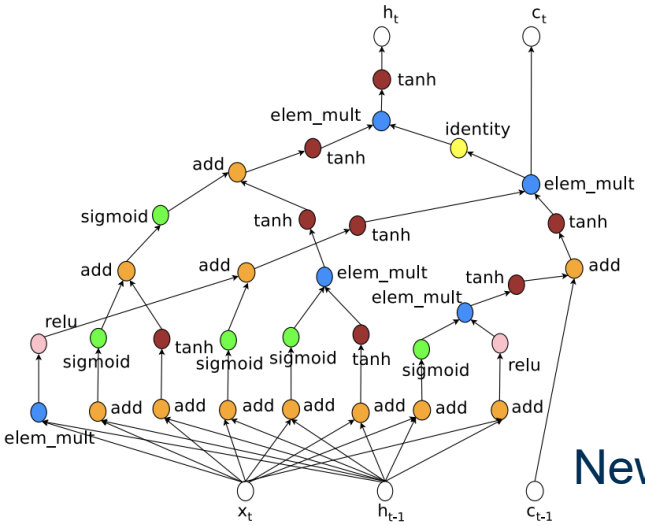
https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

Georgia Tech

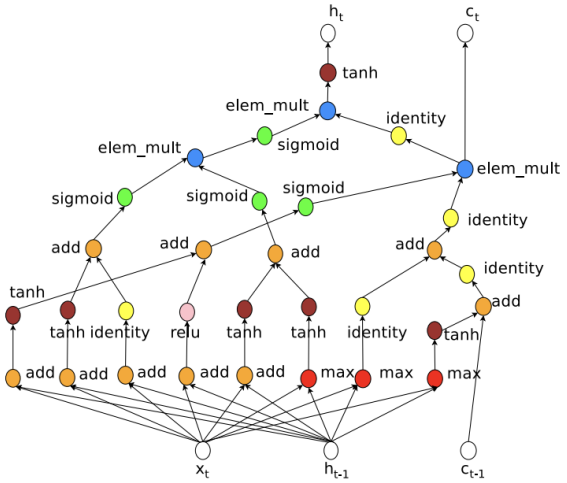
Other RNN Architectures



LSTM



New architecture 1



New architecture 2

Neural Architecture Search with Reinforcement Learning

Barret Zoph,* Quoc V. Le
Google Brain
{barretzoph,qvl}@google.com

Georgia Tech

Summary

- RNN allows us to process sequence data
- Vanilla RNN works but may have instability issue (vanishing/exploding gradient)
- LSTM and GRU improve gradient flow
- Controlling gradient in RNN/LSTM/GRU is important
  - Gradient clipping
- Architecture search is still an active research direction (nowadays LLMs as well)

Georgia Tech